Manual de usuario de 3DAlien

1. Introducción

3DAlien es una aplicación de escritorio para escribir, analizar y compilar programas de tres lenguajes:

- Pig Latin, con extensión `.pig`.
- Y?, con extensión `.y`.
- Zetariano, con extensión `.z`.

La aplicación permite abrir un proyecto, editar sus archivos, compilar el archivo principal, consultar reportes del compilador, generar código C y generar código ensamblador RISC-V ejecutable mediante RARS.

2. Requisitos

Para ejecutar la aplicación se necesita:

- Java 21 o superior.
- Sistema operativo con entorno gráfico.
- El archivo JAR generado por el proyecto.

Para compilar el proyecto desde el código fuente se necesita adicionalmente:

- JDK 21 o superior.
- Maven 3.9 o superior.
- El archivo local `Alien_code/libs/rars.jar`.

El compilador C es opcional. Sólo se necesita si se desea compilar el código C generado desde la aplicación.

3. Inicio de la aplicación

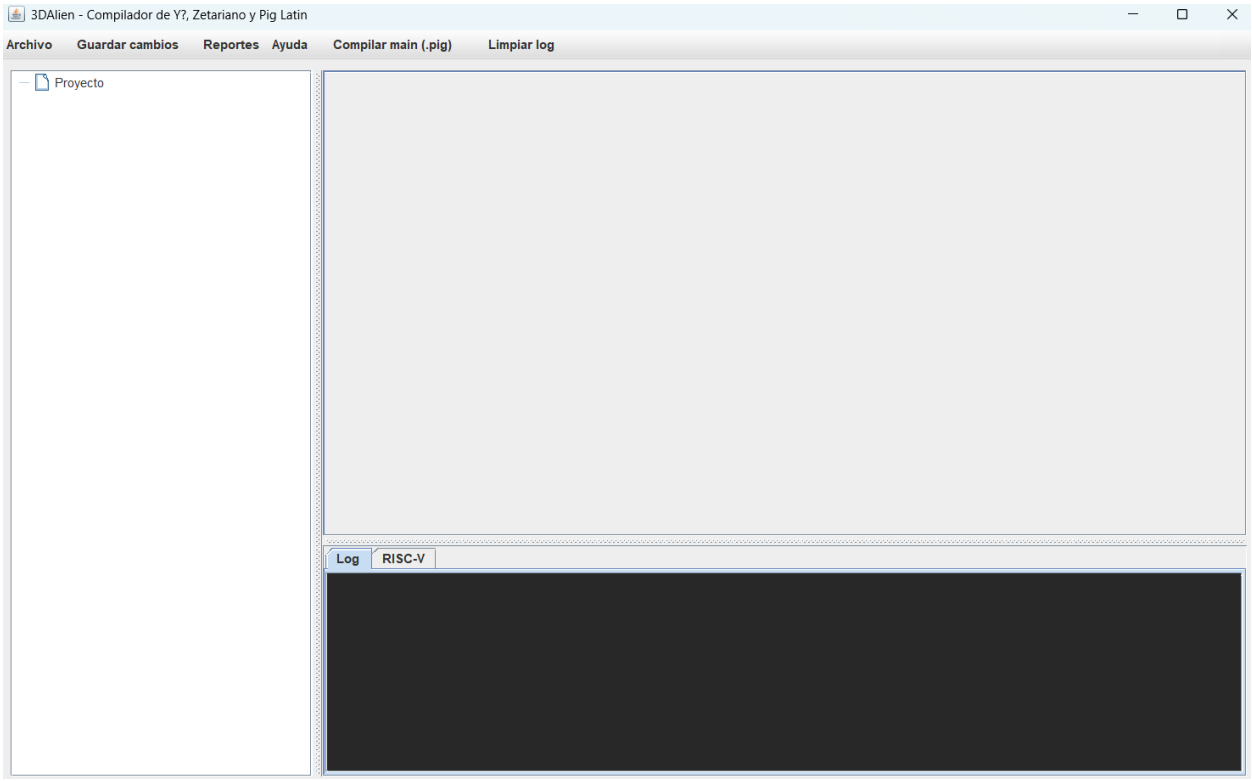
El archivo ejecutable principal se encuentra en:

```
Alien_code/target/Alien_code-1.0-SNAPSHOT.jar
```

Puede abrirse haciendo doble clic sobre el archivo o mediante una terminal:

```
java -jar Alien_code/target/Alien_code-1.0-SNAPSHOT.jar
```

Al iniciar la aplicación se muestra la ventana principal de 3DAlien.



4. Abrir un proyecto

1. Abrir el menú **Archivo**.
2. Seleccionar **Abrir carpeta de proyecto**.
3. Elegir la carpeta que contiene los archivos del programa.
4. Presionar **Seleccionar carpeta** o el botón equivalente del explorador del sistema.

Después de seleccionar la carpeta:

- El árbol de trabajo muestra los archivos y subcarpetas.
- La barra de título muestra el nombre del proyecto.
- El log informa la ubicación del proyecto abierto.

La carpeta seleccionada se mantiene como raíz visual del árbol. Esto permite conservar visible todo el proyecto aunque el archivo `.pig` principal se encuentre dentro de una subcarpeta.

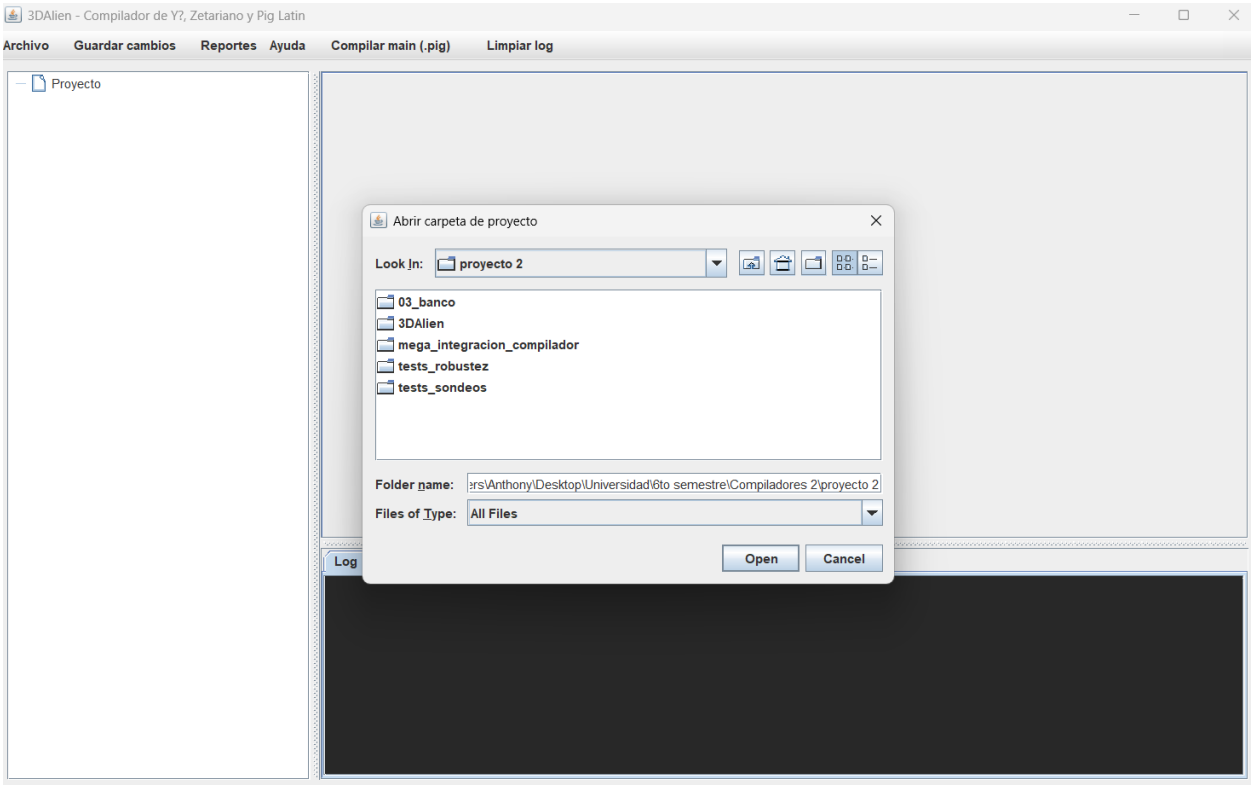
Los imports del programa se resuelven desde la carpeta que contiene el archivo principal activo. Por ejemplo, si el archivo está en:

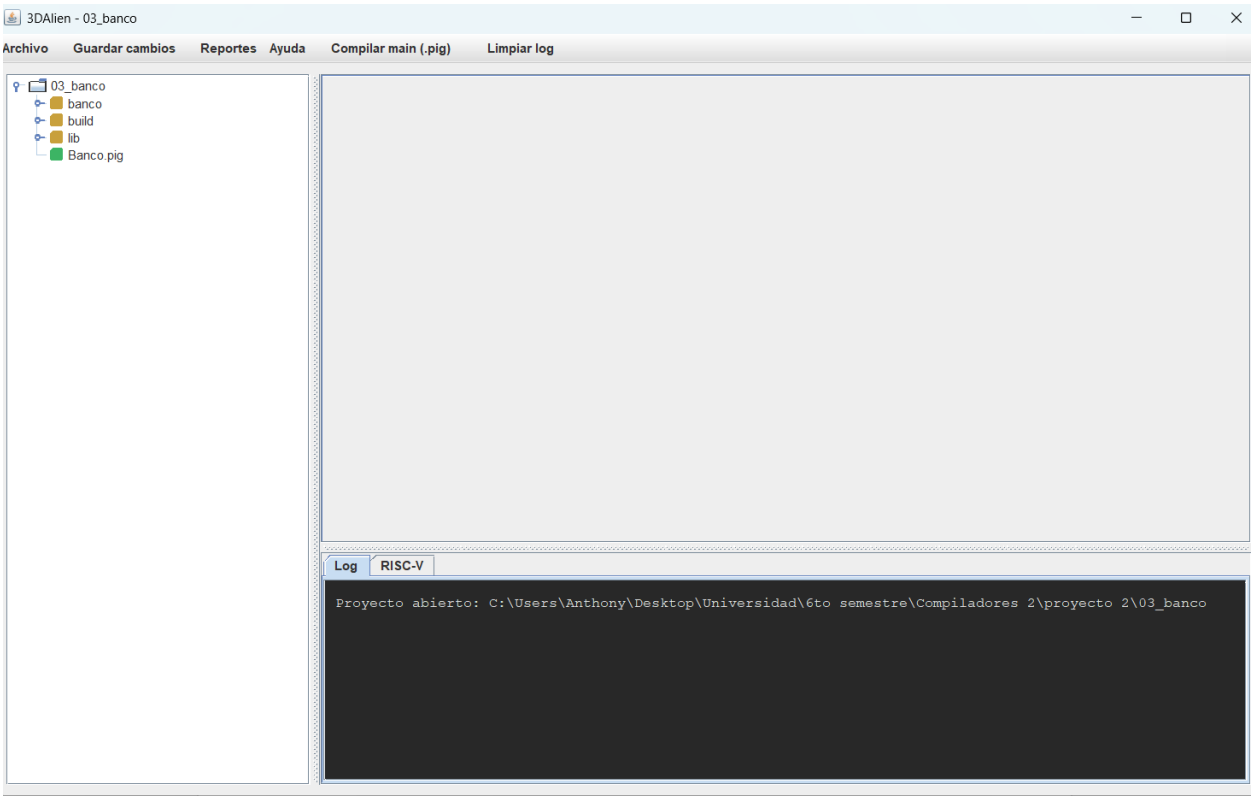
```
mi_proyecto/
└── programa/
    ├── Main.pig
    ├── lib/
    │   └── Utiles.y
    └── clases/
        └── Menu.z
```

El import:

```
import lib.Utiles.y
```

se busca dentro de `programa/lib/Utiles.y`.





5. Árbol de trabajo

El árbol de trabajo aparece en el panel izquierdo de la ventana.

5.1 Abrir archivos

Para abrir un archivo:

1. Buscarlo en el árbol.
2. Hacer doble clic sobre el archivo.
3. El archivo se abrirá en una pestaña del editor.

Los archivos de código reconocidos por el proyecto son:

Extensión	Lenguaje
`.pig`	Pig Latin
`.y`	Y?
`.z`	Zetariano

5.2 Expandir y contraer carpetas

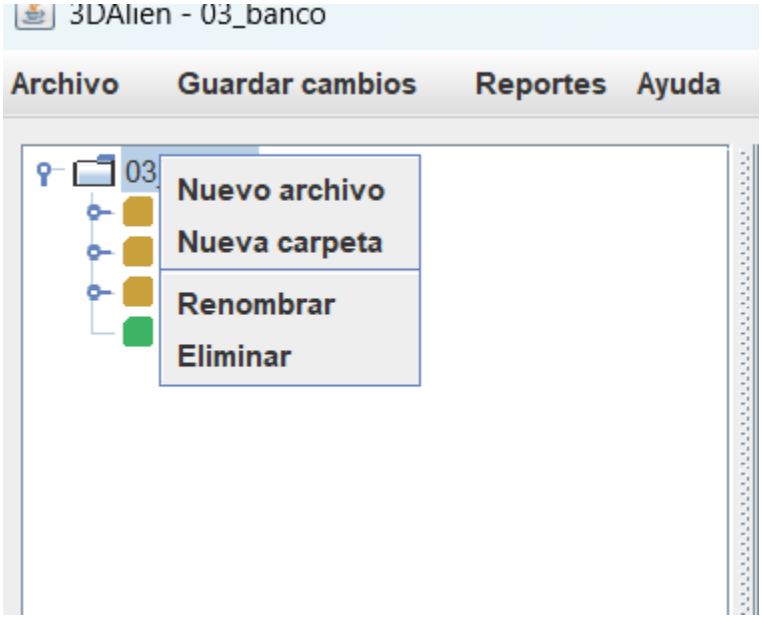
- Hacer clic sobre la flecha de una carpeta para expandirla.
- Hacer clic nuevamente para contraerla.
- Después de crear, renombrar o eliminar elementos, el árbol se recarga conservando las carpetas que estaban expandidas.
- Después de compilar, el árbol permanece abierto y conserva la raíz del proyecto.

5.3 Menú contextual

Al hacer clic derecho sobre una carpeta o archivo se muestra un menú contextual con las opciones:

- **Nuevo archivo**.
- **Nueva carpeta**.
- **Renombrar**.
- **Eliminar**.

Las operaciones se aplican sobre el elemento seleccionado o sobre la carpeta correspondiente.



6. Crear archivos y carpetas

6.1 Crear un archivo desde el menú principal

1. Abrir **Archivo**.
2. Seleccionar **Nuevo archivo**.
3. Indicar el nombre del archivo.
4. Elegir la extensión:
   - `.pig`
   - `.y`
   - `.z`
5. Confirmar.

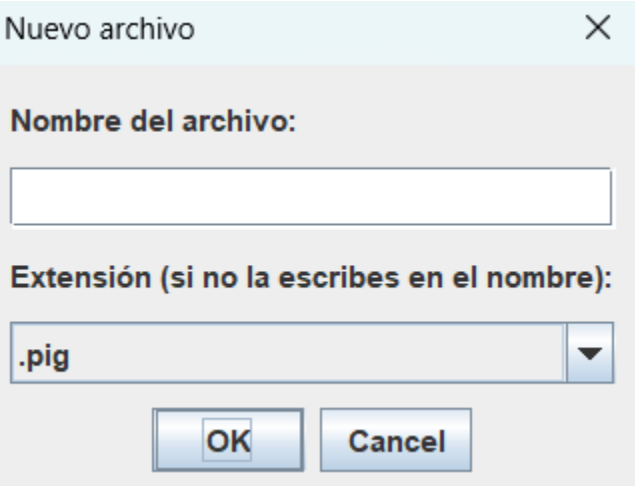
El archivo se crea dentro de la carpeta del proyecto y se abre automáticamente en el editor.

6.2 Crear un archivo desde el árbol

1. Hacer clic derecho sobre la carpeta de destino.
2. Seleccionar **Nuevo archivo**.
3. Escribir el nombre.
4. Confirmar.

6.3 Crear una carpeta

1. Hacer clic derecho sobre la carpeta donde se desea crear la nueva carpeta.
2. Seleccionar **Nueva carpeta**.
3. Escribir el nombre.
4. Confirmar.



7. Editor de código

Al abrir un archivo, el contenido se muestra en una pestaña del editor.

El editor permite:

- Escribir y modificar código.
- Trabajar con varias pestañas abiertas.
- Consultar archivos de diferentes lenguajes del mismo proyecto.
- Utilizar resaltado de sintaxis según la extensión.

Antes de compilar, se recomienda guardar los cambios del archivo activo.

8. Guardar cambios

Existen dos formas de guardar el archivo abierto:

Opción 1: Menú Archivo

1. Abrir **Archivo**.
2. Seleccionar **Guardar**.

Opción 2: Botón directo

Presionar el botón **Guardar cambios** ubicado en la barra superior.

El log informa el nombre del archivo guardado.

Es importante guardar los cambios antes de compilar porque el compilador utiliza el contenido guardado del archivo principal y de sus imports.



9. Estructura mínima de un programa Pig Latin

El archivo principal debe tener extensión `.pig` y contener la sección `MAIOR>` y la marca final `FINIS;`.

Ejemplo:

```
VARIABILES>
esto mensaje : textum "Hola";

MAIOR>
>> mensaje;
FINIS;
```

Un programa puede importar funciones, estructuras o clases:

```
import lib.Utiles.y
import clases.Menu.z
```

La ruta del import se escribe con puntos y corresponde a una ruta relativa a la carpeta del archivo `.pig`.

10. Compilar el programa principal

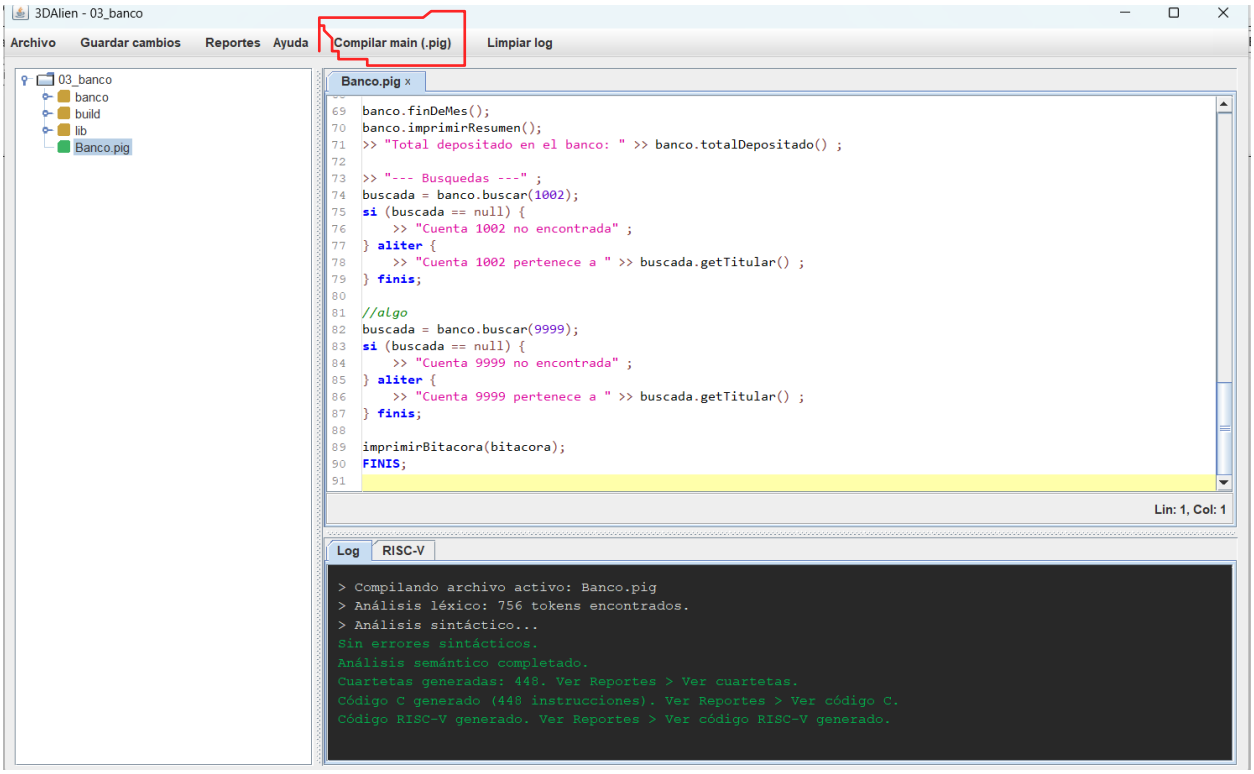
Para compilar:

1. Abrir el archivo principal `.pig`.
2. Verificar que el archivo contiene `MAIOR>`.
3. Guardar los cambios.
4. Presionar **Compilar main (.pig)**.
5. Esperar el resultado en el panel **Log**.

La compilación realiza las siguientes etapas:

1. Análisis léxico.
2. Análisis sintáctico.
3. Construcción del AST.
4. Análisis semántico.
5. Generación de cuartetas.
6. Generación de código C.
7. Generación de código RISC-V.

Si la compilación es correcta, el log muestra mensajes de éxito y se actualizan los reportes y el código generado.



11. Interpretar el panel Log

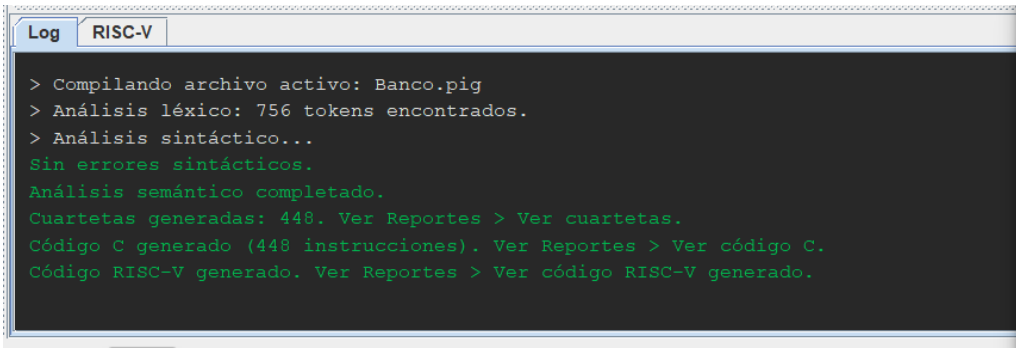
El panel **Log** muestra información del proceso de compilación y ejecución.

Los mensajes pueden indicar:

- Cantidad de tokens encontrados.
- Resultado del análisis sintáctico.
- Resultado del análisis semántico.
- Cantidad de cuartetas generadas.
- Ubicación de archivos generados.
- Resultado de la ejecución en RARS.
- Errores encontrados.

Para limpiar el contenido del log:

1. Presionar el botón **Limpiar log**.



12. Consultar errores

Para consultar los errores de la última compilación:

1. Abrir el menú **Reportes**.
2. Seleccionar **Ver errores**.

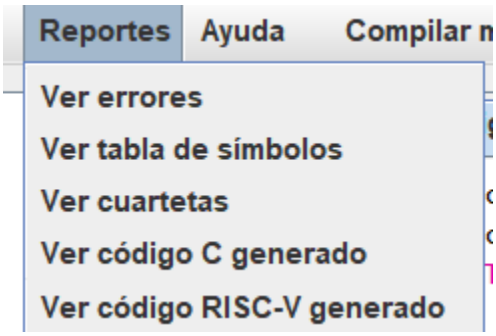
La tabla muestra:

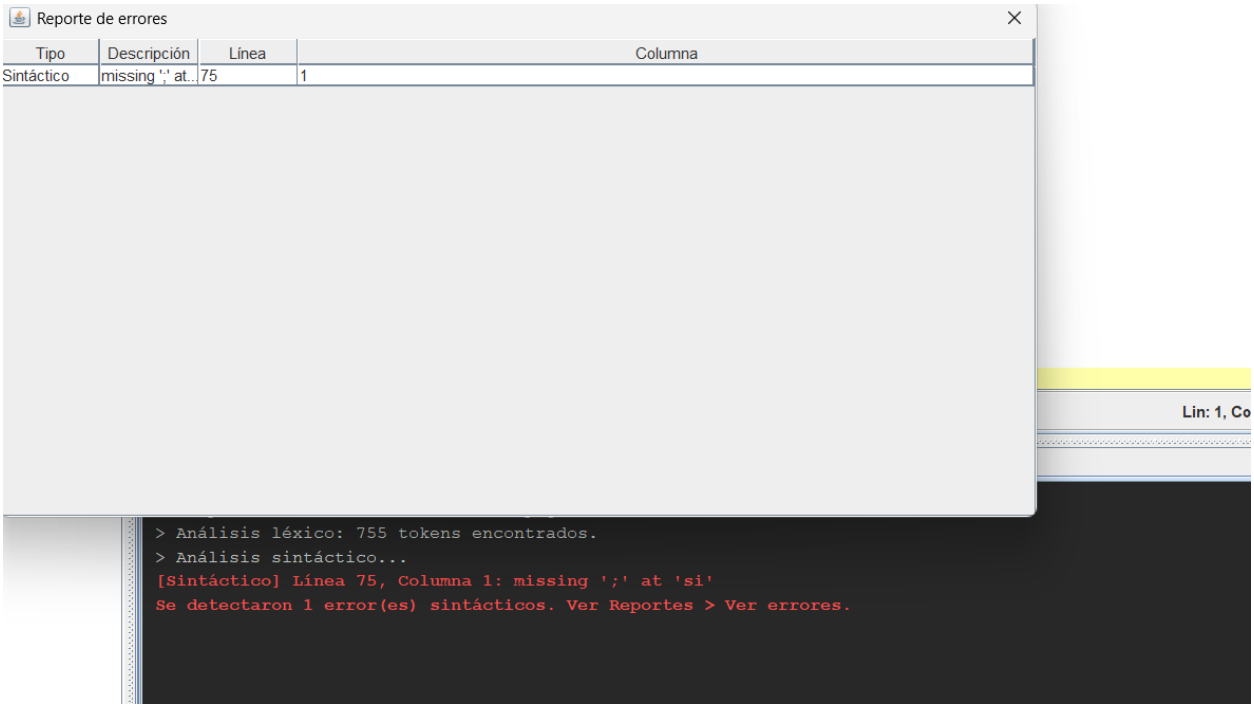
- Tipo de error.
- Descripción.
- Línea.
- Columna.

Los errores pueden ser:

- Léxicos.
- Sintácticos.
- Semánticos.

Se recomienda corregir los errores de arriba hacia abajo, porque un error inicial puede provocar otros mensajes derivados.





13. Consultar la tabla de símbolos

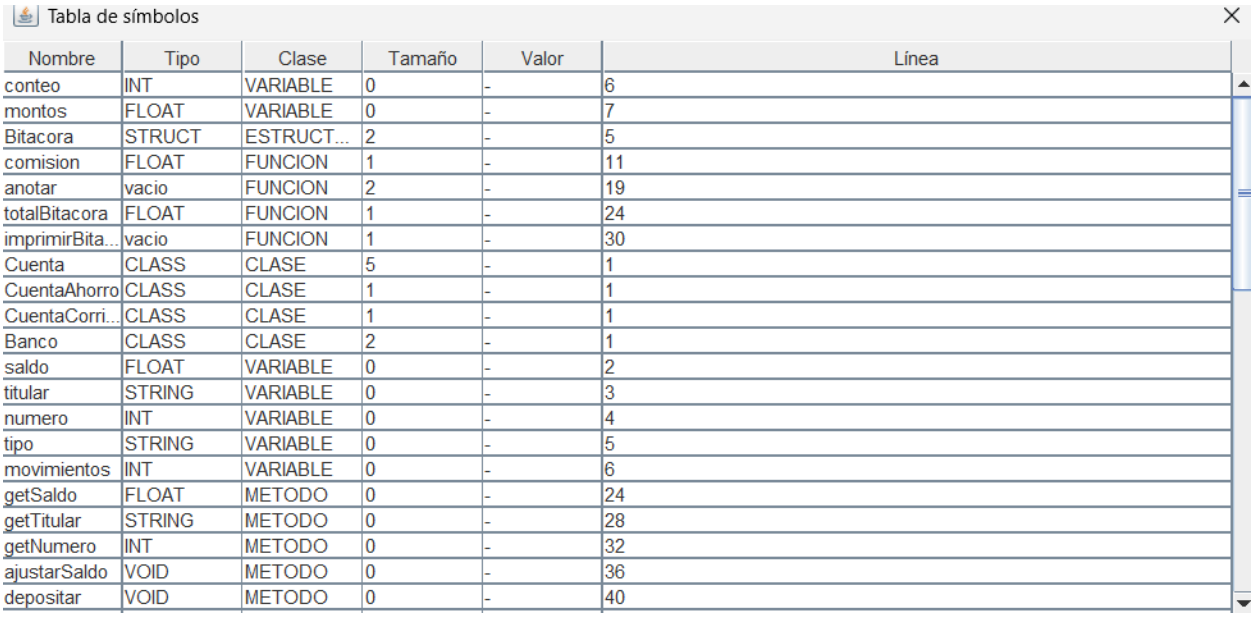
Para abrir la tabla de símbolos:

1. Abrir **Reportes**.
2. Seleccionar **Ver tabla de símbolos**.

La tabla muestra información como:

- Nombre del símbolo.
- Tipo.
- Clase de símbolo.
- Tamaño.
- Valor.
- Línea de declaración.

Este reporte sirve para verificar que las variables, funciones, estructuras y clases fueron registradas correctamente.



Nombre	Tipo	Clase	Tamaño	Valor	Línea
conteo	INT	VARIABLE	0	-	6
montos	FLOAT	VARIABLE	0	-	7
Bitacora	STRUCT	ESTRUCT...	2	-	5
comision	FLOAT	FUNCION	1	-	11
anotar	vacio	FUNCION	2	-	19
totalBitacora	FLOAT	FUNCION	1	-	24
imprimirBita...	vacio	FUNCION	1	-	30
Cuenta	CLASS	CLASE	5	-	1
CuentaAhorro	CLASS	CLASE	1	-	1
CuentaCorri...	CLASS	CLASE	1	-	1
Banco	CLASS	CLASE	2	-	1
saldo	FLOAT	VARIABLE	0	-	2
titular	STRING	VARIABLE	0	-	3
numero	INT	VARIABLE	0	-	4
tipo	STRING	VARIABLE	0	-	5
movimientos	INT	VARIABLE	0	-	6
getSaldo	FLOAT	METODO	0	-	24
getTitular	STRING	METODO	0	-	28
getNumero	INT	METODO	0	-	32
ajustarSaldo	VOID	METODO	0	-	36
depositar	VOID	METODO	0	-	40

14. Consultar las cuartetas

Para consultar el código intermedio:

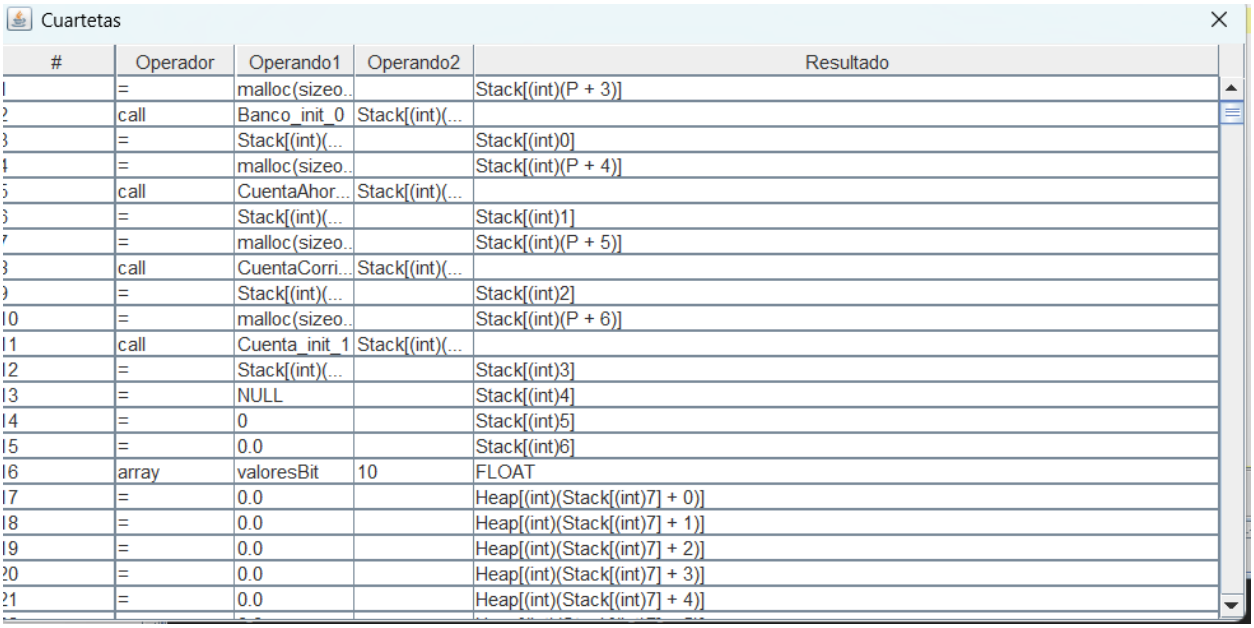
1. Abrir **Reportes**.
2. Seleccionar **Ver cuartetas**.

Las cuartetas muestran las instrucciones intermedias que utiliza el compilador antes de generar C o RISC-V.

Las columnas principales son:

- Número.
- Operador.
- Operando 1.
- Operando 2.
- Resultado.

Este reporte es útil para revisar operaciones, saltos, llamadas, asignaciones y accesos a memoria.



15. Ver y guardar código RISC-V

Para ver el ensamblador generado:

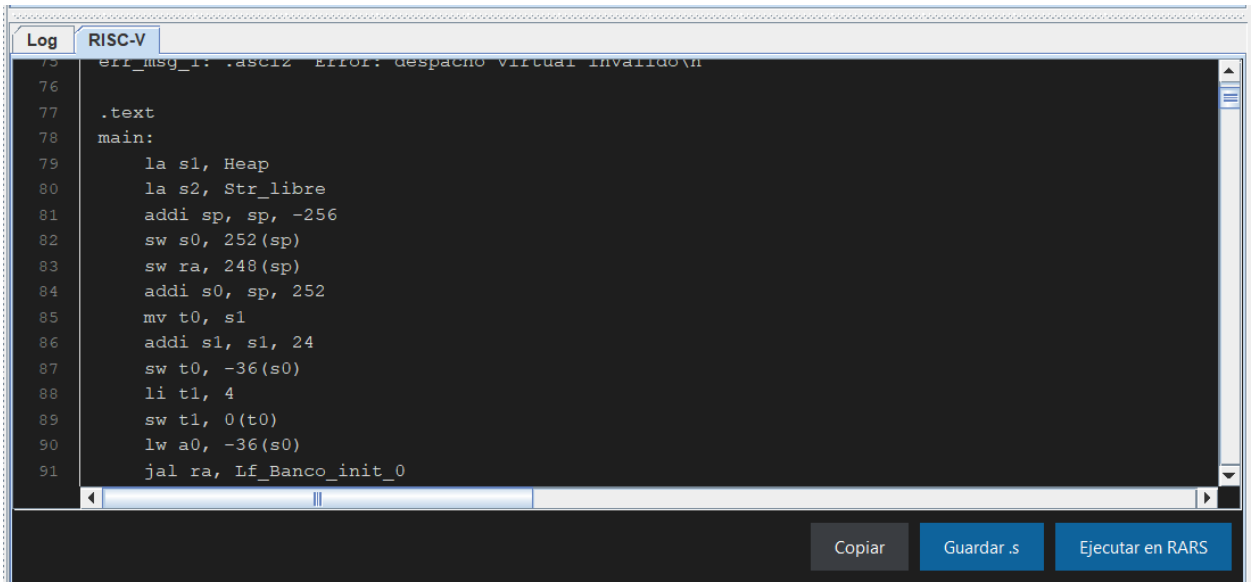
1. Compilar correctamente el programa.
2. Abrir **Reportes**.
3. Seleccionar **Ver código RISC-V generado**.

El panel RISC-V aparece como una pestaña inferior de la ventana principal.

El panel contiene los botones:

- **Copiar**: copia todo el ensamblador al portapapeles.
- **Guardar .s**: guarda el código en un archivo `.s`.
- **Ejecutar en RARS**: ensambla y ejecuta el código mediante el simulador integrado.

Las operaciones necesarias se generan directamente dentro del ensamblador producido. No es necesario agregar manualmente un archivo runtime separado.



16. Ejecutar el programa en RARS

Para ejecutar el código RISC-V:

1. Compilar el programa.
2. Abrir **Reportes > Ver código RISC-V generado**.
3. Presionar **Ejecutar en RARS**.
4. Revisar la salida en el panel **Log**.

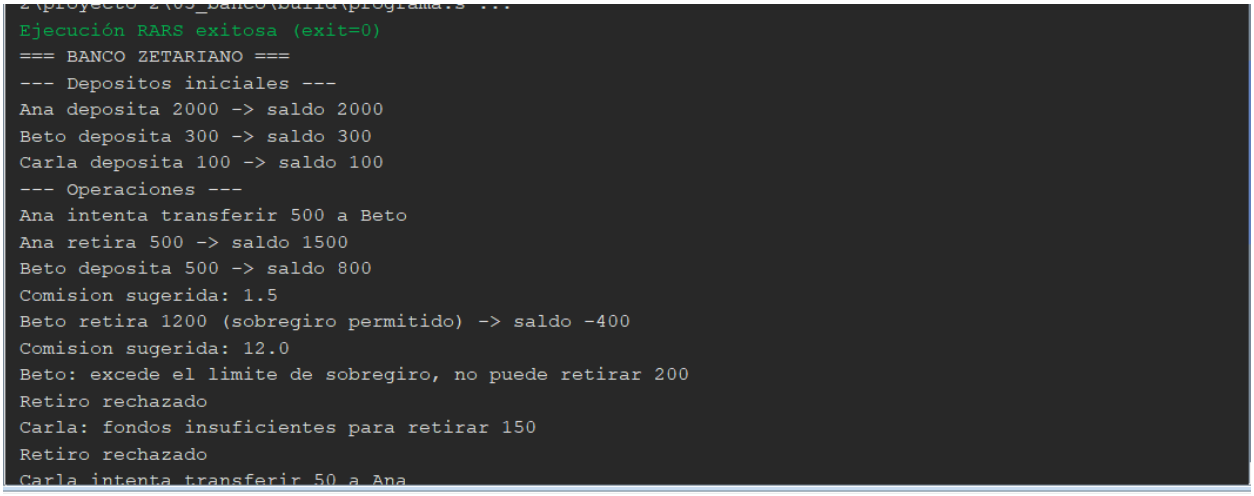
Durante la ejecución:

- El archivo se guarda dentro de la carpeta `build` del proyecto.
- RARS ensambla el programa.
- Se ejecuta el programa.
- El resultado aparece en el log.

El log puede mostrar:

- Ejecución exitosa.
- Código de salida.
- Salida impresa por el programa.
- Error de ensamblado.
- Error durante la ejecución.
- Límite de instrucciones alcanzado.

La ejecución desde el botón de la interfaz utiliza una entrada vacía. Para programas que leen datos, se recomienda utilizar una ejecución que permita proporcionar la entrada antes de iniciar RARS.



17. Ver y guardar código C

Para ver el código C:

1. Compilar correctamente el programa.
2. Abrir **Reportes**.
3. Seleccionar **Ver código C generado**.

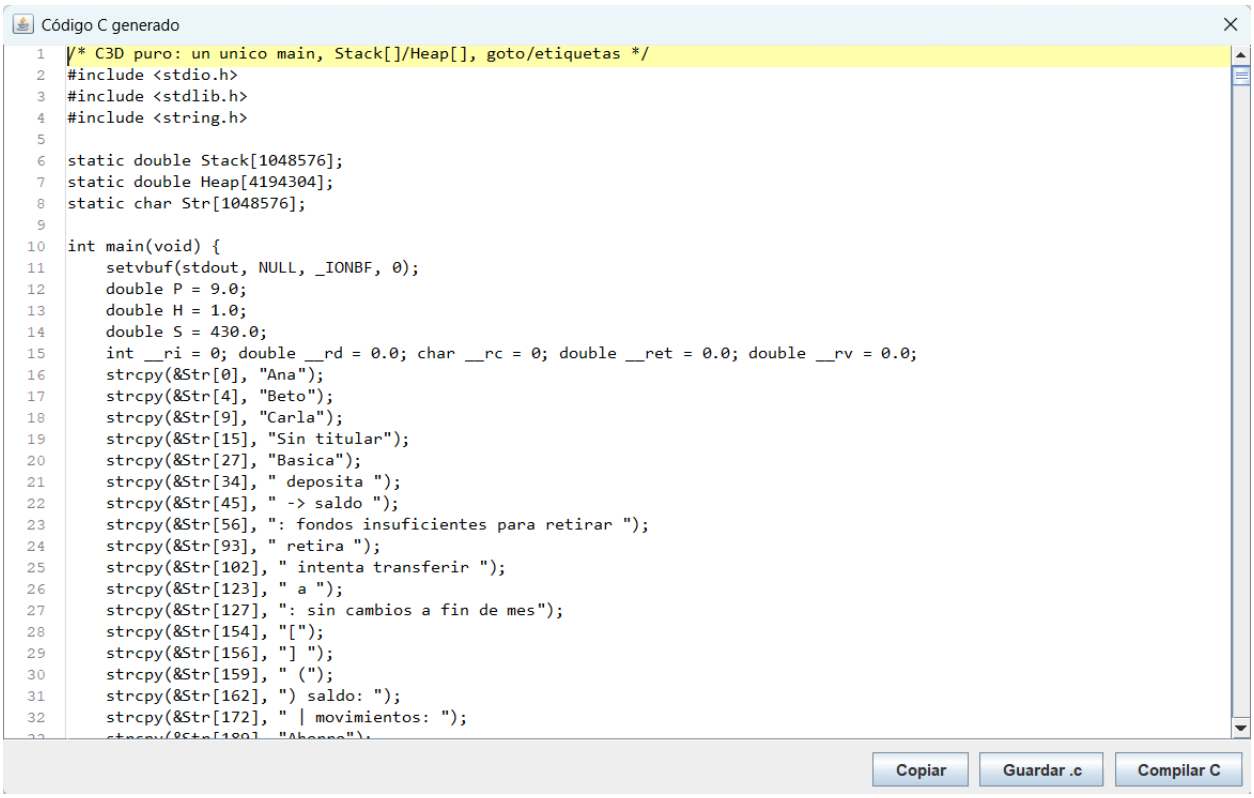
La ventana de código C contiene los botones:

- **Copiar**: copia el código al portapapeles.
- **Guardar .c**: guarda el código en un archivo C.
- **Compilar C**: intenta compilar el archivo utilizando un compilador instalado.

La aplicación busca uno de los siguientes compiladores:

- `gcc`
- `clang`
- `cl.exe`

Si no encuentra ninguno, la aplicación informa el error en el log.



18. Compilar código C

Para compilar el código C generado:

1. Verificar que GCC, Clang o `cl.exe` esté instalado.
2. Verificar que el compilador esté disponible en el `PATH`.
3. Abrir **Reportes > Ver código C generado**.
4. Presionar **Compilar C**.
5. Revisar el resultado en el panel Log.

El archivo C se guarda en:

build/programa.c

El ejecutable generado se guarda en la misma carpeta `build`.

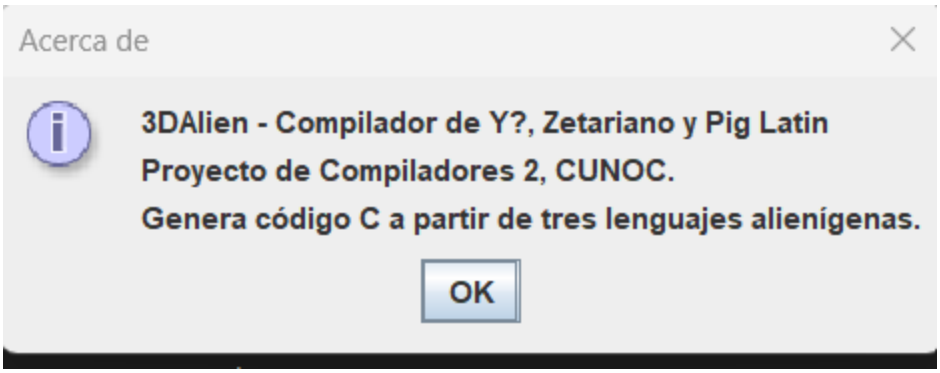
En Windows, normalmente se genera:

build/programa.exe

19. Menú Ayuda

El menú **Ayuda** contiene la opción **Acerca de**.

Esta opción muestra información general de la aplicación y del proyecto.



20. Problemas frecuentes

20.1 No aparece el archivo principal

Verificar que:

- Se abrió la carpeta correcta.
- El archivo tiene extensión `.pig`.
- El archivo aparece en el árbol.

20.2 El botón de compilación indica que no hay archivo abierto

Abrir el archivo `.pig` haciendo doble clic sobre él en el árbol.

20.3 El archivo no se puede compilar

Verificar que:

- El archivo activo tenga extensión `.pig`.
- Contenga la sección `MAIOR>`.
- Termine correctamente con `FINIS;`.
- No tenga errores sintácticos.

20.4 No se encuentra un import

Verificar que:

- El archivo importado exista.
- La ruta escrita utilice puntos.
- La ruta sea relativa a la carpeta que contiene el `.pig`.
- El nombre del archivo coincida con el nombre utilizado en el import.

Ejemplo:

import lib.Utiles.y

debe corresponder a:

lib/Utiles.y

20.5 El árbol parece cerrarse después de compilar

La versión actual conserva la raíz completa y las carpetas expandidas. Si el problema aparece:

1. Verificar que se abrió una carpeta de proyecto.
2. Volver a abrir la carpeta mediante **Archivo > Abrir carpeta de proyecto**.
3. Confirmar que el archivo `.pig` pertenece al proyecto seleccionado.

20.6 RARS muestra un error de ensamblado

Revisar:

- El mensaje del panel Log.
- El código RISC-V generado.
- Las etiquetas y saltos indicados por el error.
- Que la compilación haya terminado correctamente.

20.7 No se puede compilar C

Verificar que `gcc`, `clang` o `cl.exe` estén instalados y disponibles en el `PATH`.

20.8 La ejecución no muestra datos introducidos por el usuario

La ejecución desde el botón **Ejecutar en RARS** utiliza una entrada vacía. Los programas que utilizan lectura deben ejecutarse mediante un flujo que permita suministrar la entrada previamente.

21. Flujo recomendado de trabajo

Para trabajar con un proyecto se recomienda seguir este orden:

1. Abrir la aplicación.
2. Abrir la carpeta raíz del proyecto.
3. Abrir el archivo principal `.pig`.
4. Revisar los imports.
5. Editar el código.
6. Presionar **Guardar cambios**.
7. Presionar **Compilar main (.pig)**.
8. Revisar el panel Log.
9. Consultar errores si la compilación falla.
10. Consultar la tabla de símbolos y las cuartetas si se necesita revisar el análisis.
11. Abrir el código RISC-V o C generado.
12. Guardar o ejecutar el código generado.

22. Ubicación de archivos generados

Los archivos generados durante el trabajo se guardan dentro de la carpeta `build` del proyecto:

```
build/
├── programa.c
├── programa.s
└── programa.exe
```

El ejecutable C sólo aparece si la compilación C termina correctamente.

23. Cierre de la aplicación

Para cerrar 3DAlien:

1. Guardar los cambios pendientes.

2. Abrir **Archivo**.
3. Seleccionar **Salir**.

También puede cerrarse la ventana mediante el botón de cierre del sistema operativo.